

# PROGRAMACIÓN III

## Resumen Completo · Unidades 5, 6 y 7

Unidad 5 Búsqueda Local

Unidad 6 CSP

Unidad 7 Naturaleza

TUIA · FCEIA · UNR · 2025

## UNIDAD 5 · BÚSQUEDA LOCAL

### 5.1 Motivación y Conceptos Generales

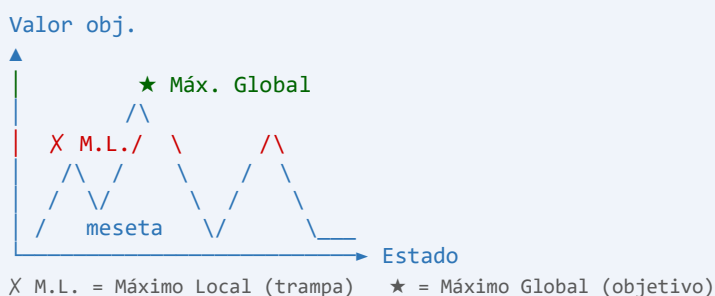
A diferencia de BFS o A\*, que mantienen en memoria **todo el árbol de caminos explorados**, una búsqueda local mantiene **solo el estado actual** (o un conjunto pequeño). No le importa el camino, solo llegar al **estado óptimo**.

#### 🔍 Diferencias clave

- **Búsqueda clásica (BFS/A\*)**: guarda árbol de caminos → mucha memoria → sirve cuando el CAMINO importa.
- **Búsqueda local**: guarda solo estado actual → poca memoria → sirve cuando solo importa el ESTADO FINAL.

### 5.2 Paisaje del Espacio de Estados

Se visualiza el problema como un paisaje topográfico. Cada punto del terreno es un estado; la altitud es el valor de la función objetivo. El algoritmo "camina" por ese terreno buscando el pico más alto (maximizar) o el valle más profundo (minimizar).



### 5.3 Búsqueda de Ascensión de Colinas (Hill-Climbing)

Es la búsqueda local más simple: en cada paso se mueve al sucesor con mayor valor objetivo. Es "greedy" (avara) porque solo mira un paso adelante.

```

Hill-Climbing básico
función HILL-CLIMBING(problema):
    actual ← ESTADO-INICIAL(problema)
    repetir:
        vecino ← sucesor con MAYOR valor objetivo de actual
        si VALOR(vecino) ≤ VALOR(actual):
            retornar actual      ← atascado (máx local o global)
        actual ← vecino
  
```

## Problemas del paisaje — ¿por qué se atasca?

| Accidente topográfico           | Descripción                                                     | ¿Tiene salida?          |
|---------------------------------|-----------------------------------------------------------------|-------------------------|
| Máximo local                    | Pico más alto que sus vecinos inmediatos, pero no es el global  | No (con HC básico)      |
| Cresta                          | Cadena de máximos locales, obliga a zigzag ineficiente          | Difícil                 |
| Meseta plana (máx. local plano) | Zona completamente plana, sin sucesor mejor NI igual con salida | No                      |
| Terraza                         | Zona plana, pero existe salida ascendente si se cruza           | Sí (con mov. laterales) |

### Movimientos Laterales

Para escapar de una **terrazza**, se permite moverse a un sucesor con el **mismo valor objetivo**. Pero hay que limitar la cantidad de movimientos laterales consecutivos con un parámetro **MAX\_LAT**, si no el algoritmo puede ciclar infinitamente.

```
hill_climbing.py
# Hill-Climbing con movimientos laterales
import random
def hill_climbing(problema, max_laterales=100):
    """
    problema: objeto con métodos:
    - estado_inicial()      -> estado
    - sucesores(estado)     -> lista de estados
    - valor_objetivo(estado) -> float (mayor = mejor)
    """
    actual = problema.estado_inicial()
    laterales = 0          # contador de movimientos laterales consecutivos

    while True:
        vecinos = problema.sucesores(actual)
        if not vecinos:
            return actual  # sin sucesores → terminamos

        # Elegir el vecino con mayor valor objetivo
        mejor_vecino = max(vecinos, key=problema.valor_objetivo)
        delta = problema.valor_objetivo(mejor_vecino) - problema.valor_objetivo(actual)

        if delta > 0:
            # Movimiento ascendente → aceptar y reiniciar contador lateral
            actual = mejor_vecino
            laterales = 0
        elif delta == 0 and laterales < max_laterales:
            # Movimiento lateral → aceptar solo si no superamos el límite
            actual = mejor_vecino
            laterales += 1
        else:
            # No hay mejora ni lateral disponible → atascado
            return actual
```

## Variantes de Hill-Climbing

| Variante           | Cómo elige el siguiente estado                           | Ventaja principal                    |
|--------------------|----------------------------------------------------------|--------------------------------------|
| Estándar (Greedy)  | Mejor sucesor absoluto                                   | Simple                               |
| Estocástico        | Aleatorio entre los que mejoran                          | Escapa crestas con zigzag            |
| Primera opción     | Primer sucesor generado que mejore                       | Eficiente con muchos sucesores       |
| Reinicio aleatorio | Múltiples corridas desde estados random; guarda el mejor | Garantiza optimalidad probabilística |

```

hill_climbing_reinicio.py
# Hill-Climbing con reinicio aleatorio
def hill_climbing_reinicio(problema, n_reinicios=50):
    """Corre n_reinicios veces y devuelve el mejor resultado global."""
    mejor_global = None
    mejor_valor = float('-inf')

    for intento in range(n_reinicios):
        # Generar un estado inicial aleatorio en cada reinicio
        problema.estado_inicial = problema.estado_aleatorio
        resultado = hill_climbing(problema) # HC básico
        valor = problema.valor_objetivo(resultado)

        if valor > mejor_valor:
            mejor_valor = valor
            mejor_global = resultado # Actualizar récord global

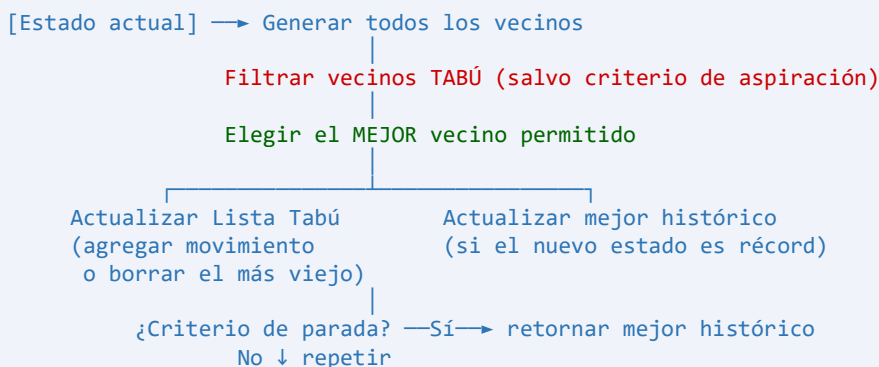
    return mejor_global

```

## 5.4 Búsqueda Tabú

Mejora HC permitiendo **moverse siempre al mejor sucesor**, aunque empeore. Para evitar ciclos, usa una **Lista Tabú**: memoria de corto plazo que prohíbe temporalmente volver a estados (o acciones) recién visitados.

### Flujo de Búsqueda Tabú



## La Lista Tabú — ¿Qué guardar?

| Qué guardar | Descripción                                    | Memoria/Tiempo | Restricción      |
|-------------|------------------------------------------------|----------------|------------------|
| Acciones    | Prohíbe la acción inversa al último movimiento | Baja           | Fuerte (directa) |
| Estados     | Guarda estados completos visitados             | Alta           | Moderada         |
| Propiedades | Guarda atributos/características del estado    | Media          | Flexible         |

## Gestión de la Lista: duración del tabú

| Estrategia                | Funcionamiento                                                                                 |
|---------------------------|------------------------------------------------------------------------------------------------|
| Capacidad limitada (FIFO) | Cola de tamaño fijo: al agregar un elemento nuevo, se borra el más viejo automáticamente       |
| Tenor de tabú             | Cada elemento tiene un contador; se elimina cuando supera el límite de iteraciones configurado |

## Componentes adicionales de Búsqueda Tabú

**Criterio de Aspiración** → Si un movimiento tabú lleva a un estado MEJOR que el mejor histórico, se acepta igual (no puede haber ciclo porque rompe un récord).

**Diversificación (memoria de largo plazo)** → Usa frecuencia de visitas para forzar exploración de zonas poco visitadas. Dos estrategias:

- De reinicio: reiniciar desde estados con componentes poco frecuentes.
- Continua: premiar sucesores con componentes poco frecuentes al elegir.

busqueda\_tabu.py

# Búsqueda Tabú simplificada

from collections import deque

```
def busqueda_tabu(problema, capacidad_tabu=10, max_iter=500):
```

```
    """
```

```
    capacidad_tabu: tamaño máximo de la lista tabú (FIFO)
```

```
    max_iter:      criterio de parada por iteraciones totales
```

```
    """
```

```
    actual      = problema.estado_inicial()
```

```
    mejor      = actual                      # mejor estado histórico
```

```
    lista_tabu = deque(maxlen=capacidad_tabu) # cola FIFO automática
```

```
    sin_mejora = 0                          # criterio parada por estancamiento
```

```
    for _ in range(max_iter):
```

```
        vecinos = problema.sucesores(actual)
```

```

# Filtrar vecinos que están en la lista tabú
# EXCEPTO si cumplen el criterio de aspiración (mejoran el récord)
candidatos = []
for v in vecinos:
    es_tabu = v in lista_tabu
    es_aspirado = problema.valor(v) > problema.valor(mejor)
    if not es_tabu or es_aspirado:
        candidatos.append(v)

if not candidatos:
    break # todos los vecinos son tabú y ninguno aspira → terminamos

# Moverse al mejor candidato (aunque empeore el estado actual)
actual = max(candidatos, key=problema.valor)
lista_tabu.append(actual) # el más viejo se borra solo (maxlen)

# Actualizar el mejor histórico
if problema.valor(actual) > problema.valor(mejor):
    mejor = actual
    sin_mejora = 0
else:
    sin_mejora += 1

if sin_mejora > 50: # criterio de parada por estancamiento
    break

return mejor

```

## 5.5 Criterios de Parada y Seteo de Parámetros

### Criterios de parada (cuándo terminar):

1. Tras N iteraciones totales (ej. max\_iter=1000)
2. Tras K iteraciones sin mejora en el valor objetivo
3. Cuando la función objetivo supera un umbral dado

⚠ **Los parámetros (capacidad, tenor, qué guardar) se determinan EXPERIMENTALMENTE.**

Un mínimo cambio puede mejorar o empeorar drásticamente la performance.

## 5.6 Búsqueda Local en Espacios Continuos

Cuando las magnitudes son continuas, los estados tienen **infinitos sucesores**: no se puede iterar sobre ellos. La solución es usar el **gradiente matemático**, que indica la dirección de mayor pendiente de la función objetivo.

### Ascenso por gradiente (Gradient Ascent):

$x_{\text{nuevo}} = x_{\text{actual}} + \alpha \cdot \nabla f(x_{\text{actual}})$

$\alpha$  = tamaño del paso (step size)     $\nabla f$  = gradiente (vector de derivadas parciales)

## UNIDAD 6 · PROBLEMAS DE SATISFACCIÓN DE RESTRICCIONES (CSP)

### 6.1 Motivación y Definición Formal

Un CSP modela **problemas de factibilidad**: no busca el mejor camino sino una **asignación de valores que cumpla todas las restricciones**. Los estados se representan como variables con dominios, no como cajas negras.

#### Componentes formales de un CSP:

$X = \{X_1, X_2, \dots, X_n\}$  Variables  
 $D = \{D_1, D_2, \dots, D_n\}$  Dominios (valores permitidos para cada  $X_i$ )  
 $C = \{C_1, C_2, \dots, C_m\}$  Restricciones (combinaciones válidas)  
 Solución = asignación **COMPLETA** (todas las variables) + **CONSISTENTE** (ninguna restricción violada)

### Clasificación de Variables y Restricciones

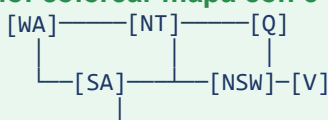
| Tipo de Variable  | Descripción                                              | Ejemplo               |
|-------------------|----------------------------------------------------------|-----------------------|
| Discreta finita   | Conjunto finito de valores posibles. Asignaciones: $d^n$ | Colores, booleanos    |
| Discreta infinita | Enteros, sin límite superior claro                       | Tiempos en scheduling |
| Continua          | Números reales – se resuelve con programación lineal     | Coordenadas GPS       |

| Tipo de Restricción | Variables involucradas  | Observación clave                                        |
|---------------------|-------------------------|----------------------------------------------------------|
| Unaria              | 1 variable              | Se elimina reduciendo el dominio directamente            |
| Binaria             | 2 variables             | Se modela como arco en el grafo de restricciones         |
| Global              | N variables arbitrarias | Toda restricción global se puede transformar en binarias |

### 6.2 Grafo de Restricciones

Se modela el CSP como un grafo donde los **nodos son variables** y los **arcos son restricciones binarias**. La topología del grafo determina la dificultad del problema.

#### Ejemplo: colorear mapa con 3 colores (R, G, B)



[T] (Tasmania: sin restricciones)

Restricción en cada arco: los nodos conectados NO pueden tener el mismo color.

## 6.3 Formulación para Búsqueda + Conmutatividad

### Formulación estándar:

Estado inicial: asignación vacía {}

Acciones: elegir variable no asignada → asignar valor consistente

Test objetivo: ¿asignación completa?

**Conmutatividad** → el orden de asignación no altera el resultado. Se asigna UNA variable por nivel del árbol, reduciendo drásticamente las ramas redundantes.

## 6.4 Backtracking Search

Es el algoritmo por defecto para CSP. DFS adaptado: **una variable por nivel**, prueba valores consistentes, y si queda sin opciones **retrocede (backtrack)** a la decisión anterior.

```

┌ Backtracking Search ───────────────────
función BACKTRACKING(asignación, csp):
  si asignación está COMPLETA:
    retornar asignación ← SOLUCIÓN

  var ← ELEGIR_VAR(csp, asignación) ← heurística de variable

  para cada valor en ORDENAR_DOM(var, csp, asignación):
    si valor es CONSISTENTE con asignación:
      agregar {var = valor} a asignación
      inferencia ← INFERIR(csp, var, valor) ← FC o AC-3
      si inferencia ≠ FALLO:
        resultado ← BACKTRACKING(asignación, csp)
        si resultado ≠ FALLO: retornar resultado
      deshacer {var = valor} y efectos de inferencia

  retornar FALLO

```

## Heurísticas de Backtracking

| Heurística                      | Qué hace                                                                          | Estrategia                    |
|---------------------------------|-----------------------------------------------------------------------------------|-------------------------------|
| MVR (Mínimos Valores Restantes) | Elige la variable con MENOS valores legales en su dominio                         | Elegir variable (ELEGIR_VAR)  |
| Grado Heurístico                | Desempata MVR: elige la variable con MÁS restricciones con variables no asignadas | Elegir variable (desempate)   |
| Valor Menos Restringido         | Prefiere el valor que elimine MENOS opciones en los dominios de sus vecinos       | Ordenar valores (ORDENAR_DOM) |

**Ejemplo: coloreado de mapa (WA=Rojo ya asignado)**  
 Antes de asignar WA: NT={R,G,B} SA={R,G,B} Q={R,G,B}  
 WA=Rojo asignado → FC elimina Rojo de vecinos de WA:  
 Después de FC: NT={G,B} SA={G,B} Q={R,G,B}

**Limitación:** FC no detecta conflictos ENTRE variables futuras (entre NT y SA en este ejemplo).

```

AC-3
función AC-3(csp):
    cola ← todos los arcos  $(X_i, X_j)$  del CSP
    mientras cola no esté vacía:
         $(X_i, X_j) \leftarrow$  extraer arco de la cola
        si REVISAR( $X_i, X_j$ ):
            ← eliminó algún valor de  $\text{Dom}(X_i)$ ?
            si  $\text{Dom}(X_i)$  está vacío:
                retornar FALLO
                ← inconsistente
            para cada  $X_k$  vecino de  $X_i$  ( $\neq X_j$ ):
                agregar  $(X_k, X_i)$  a la cola ← propagar a los vecinos
    retornar ÉXITO

función REVISAR( $X_i, X_j$ ):
    eliminado ← falso
    para cada  $x$  en  $\text{Dom}(X_i)$ :
        si NO existe  $y$  en  $\text{Dom}(X_j)$  tal que  $(x,y)$  satisface restricción:
            eliminar  $x$  de  $\text{Dom}(X_i)$ 
            eliminado ← verdadero
    retornar eliminado

```

| Inferencia       | Alcance                                       | Costo | Detecta conflictos futuros    |
|------------------|-----------------------------------------------|-------|-------------------------------|
| Forward Checking | Solo vecinos directos de la variable asignada | Bajo  | No entre variables futuras    |
| AC-3             | Todo el grafo (propagación en cadena)         | Mayor | Sí, entre todas las variables |

## 6.6 Restricciones Globales Especiales

### Alldiff — todas las variables deben tener valores distintos

Test rápido de inconsistencia: si hay  $n$  variables y sus dominios combinados tienen solo  $m$  valores posibles, y  $m < n \rightarrow$  FALLO inmediato (imposible asignar distintos a todos).

### Atmost — la suma de valores no debe superar un límite

Test rápido: si la suma de los valores MÍNIMOS posibles en cada dominio ya supera el límite  $\rightarrow$  FALLO inmediato.

## 6.7 Backjumping — Vuelta Atrás Inteligente

El backtracking estándar siempre sube UN nivel (vuelta cronológica), lo que puede repetir fallos inútilmente. El **Salto-atrás dirigido por conflicto** identifica el **conjunto conflicto** (variables asignadas que causaron el fallo) y salta directamente a la más reciente, ignorando variables intermedias irrelevantes.

### Backtracking cronológico vs Backjumping:

Árbol:  $X1 \rightarrow X2 \rightarrow X3 \rightarrow X4 \rightarrow X5 \rightarrow \text{FALLO}$

**Backtracking cronológico:** sube a  $X4$ , luego  $X3$ ,  $X2...$  (paso a paso)

**Backjumping:** el fallo depende de  $X1$  y  $X3 \rightarrow$  salta directo a  $X3$  (omite  $X4$ )

## 6.8 Resolución con Búsqueda Local — Min-Conflicts

Los algoritmos locales operan sobre **estados completos (todas las variables ya asignadas)**, aunque violen restricciones. El objetivo es minimizar el número de restricciones violadas.

```

┌ Min-Conflicts ────────────────────
función MIN-CONFLICTS(csp, max_pasos):
  asignación ← asignación ALEATORIA completa (puede ser inconsistente)
  para i = 1 hasta max_pasos:
    si asignación es CONSISTENTE:
      retornar asignación      ← SOLUCIÓN
    var ← elegir ALEATORIAMENTE una variable en conflicto
    val ← valor de Dom(var) que MINIMIZA restricciones violadas
    asignar var = val en asignación
  retornar FALLO
└───────────────────────────────────

```

```

❏ min_conflicts_reinas.py
# Min-Conflicts para N-Reinas
import random

def contar_conflictos(tablero, col, fila):
    """Cuántas reinas atacan a la reina en (col, fila)."""
    n = len(tablero)
    conflictos = 0
    for c in range(n):
        if c == col:
            continue
        f = tablero[c]
        # Misma fila o misma diagonal
        if f == fila or abs(f - fila) == abs(c - col):
            conflictos += 1
    return conflictos

def min_conflicts_reinas(n, max_pasos=10000):
    """
    Resuelve N-Reinas con Min-Conflicts.
    tablero[col] = fila donde está la reina en esa columna.
    """
    # Estado inicial: asignación aleatoria completa
    tablero = [random.randint(0, n-1) for _ in range(n)]

    for paso in range(max_pasos):
        # Detectar columnas con conflictos
        en_conflicto = [c for c in range(n) if contar_conflictos(tablero, c, tablero[c]) > 0]

        if not en_conflicto:
            return tablero    # Solución encontrada

        # Elegir una columna en conflicto al azar
        col = random.choice(en_conflicto)

        # Elegir la fila que minimiza los conflictos para esa columna
        fila_min = min(range(n), key=lambda f: contar_conflictos(tablero, col, f))
        tablero[col] = fila_min

    return None    # No encontró solución en max_pasos

```

## 6.9 Estructura del Problema — Componentes Conexas

La dificultad de un CSP depende de la **topología del grafo de restricciones**. Si el grafo se puede dividir en componentes independientes (sin restricciones cruzadas), la complejidad cae drásticamente:

### Impacto de los subproblemas independientes:

Problema entero (n variables, d valores):  $d^n$  asignaciones posibles

Dividido en p subproblemas (n/p vars c/u):  $p \cdot d^{(n/p)}$  asignaciones

Ejemplo: n=80, d=2, p=4 → de  $2^{80}$  a  $4 \cdot 2^{20}$  (diferencia de ASTRONOMÍA)

- Mecánicas estocásticas (aleatorias) → sacrifican sistematismo por eficiencia en paisajes masivos.
- Alta efectividad en problemas de optimización grandes.
- La mayoría son búsquedas basadas en POBLACIONES (múltiples estados simultáneos).

Excepción: SA.

```

❏ simulated_annealing.py
# Recocido Simulado
import math, random

def recocido_simulado(problema, T_inicial=1000, alpha=0.995, T_min=0.01):
    """
    T_inicial: temperatura de arranque (alta = acepta muchos movimientos malos)
    alpha:     factor de enfriamiento (ej. 0.995 → temperatura baja un 0.5% por iteración)
    T_min:     temperatura de parada
    """
    mejor = actual
    T = T_inicial

    while T > T_min:
        vecino = problema.sucesor_aleatorio(actual)
        delta = problema.valor(vecino) - problema.valor(actual)

        if delta > 0:
            # El vecino es mejor → aceptar siempre
            actual = vecino
        else:
            # El vecino es peor → aceptar con probabilidad  $e^{(\Delta E/T)}$ 
            prob = math.exp(delta / T)
            if random.random() < prob:
                actual = vecino # movimiento "malo" aceptado

        # Actualizar el mejor histórico
        if problema.valor(actual) > problema.valor(mejor):
            mejor = actual
        T *= alpha # Enfriar temperatura
    return mejor

```

## 7.2 Algoritmo Genético (Genetic Algorithm — GA)

Inspirado en la **selección natural de Darwin**. Evoluciona una población de soluciones candidatas a través de selección, cruzamiento y mutación.

### Ciclo de un Algoritmo Genético:

Población inicial (individuos aleatorios)

1. Evaluar fitness de cada individuo
2. Seleccionar pares (prob. proporcional al fitness)
3. Cruzamiento (crossover) → nuevos hijos
4. Mutación aleatoria (prob. baja) en los hijos
5. Nueva generación reemplaza a la anterior

¿Criterio de parada? → No: volver al paso 1

## Fase 4: Cruzamiento (Crossover)

### Ejemplo: cruce de un punto en strings de bits

Padre A: [1 1 0 0 1 | 0 1 1 0]

Padre B: [0 0 1 1 0 | 1 0 0 1]

Punto de corte: posición 5

Hijo 1: [1 1 0 0 1] + [1 0 0 1] = [1 1 0 0 1 1 0 0 1]

Hijo 2: [0 0 1 1 0] + [0 1 1 0] = [0 0 1 1 0 0 1 1 0]

⚠ **Problema en TSP:** el cruce clásico puede generar rutas ilegales (ciudades repetidas). Se usa representación ordinal u operadores de cruce parciales.

```

⊗ algoritmo_genetico.py
# Algoritmo Genético básico
import random

def inicializar_poblacion(tam_poblacion, longitud_gen):
    """Generar individuos aleatorios (strings de bits)."""
    return [[random.randint(0, 1) for _ in range(longitud_gen)]
            for _ in range(tam_poblacion)]

def seleccionar(poblacion, fitness_fn):
    """Selección por ruleta: probabilidad proporcional al fitness."""
    fitnesses = [fitness_fn(ind) for ind in poblacion]
    total = sum(fitnesses)
    # Elegir con probabilidad proporcional
    r = random.uniform(0, total)
    acum = 0
    for ind, f in zip(poblacion, fitnesses):
        acum += f
        if acum >= r:
            return ind

def cruzar(padre_a, padre_b):
    """Cruzamiento de un punto: cortar y mezclar los genes."""
    punto = random.randint(1, len(padre_a) - 1)
    hijo1 = padre_a[:punto] + padre_b[punto:]
    hijo2 = padre_b[:punto] + padre_a[punto:]
    return hijo1, hijo2

def mutar(individuo, prob_mutacion=0.01):
    """Mutación: invertir bit con probabilidad baja."""
    return [gen if random.random() > prob_mutacion else 1 - gen
            for gen in individuo]

def algoritmo_genetico(fitness_fn, longitud_gen, generaciones=200,
                      tam_poblacion=50, prob_mutacion=0.01):
    poblacion = inicializar_poblacion(tam_poblacion, longitud_gen)
    for gen in range(generaciones):
        nueva_poblacion = []
        # Retornar el mejor individuo de la última generación
    return max(poblacion, key=fitness_fn)

```

## 7.3 Optimización por Colonia de Hormigas (ACO)

Inspirado en cómo las hormigas encuentran caminos cortos usando **feromonas**. Las hormigas artificiales construyen soluciones probabilísticamente y depositan más feromona en los caminos cortos.

### Probabilidad de elegir arco (i→j) en TSP:

$$P(i \rightarrow j) = \frac{[\tau(i,j)]^\alpha \cdot [1/d(i,j)]^\beta}{\sum_k [\tau(i,k)]^\alpha \cdot [1/d(i,k)]^\beta}$$

$\tau(i,j)$  = feromona en la arista → rutas más transitadas son más atractivas

$d(i,j)$  = distancia real → rutas más cortas son más atractivas

### Ciclo de Feromonas:

1. Cada hormiga construye una ruta completa (probabilísticamente)

2. **EVAPORACIÓN:**  $\tau(i,j) \leftarrow (1-\rho) \cdot \tau(i,j)$  [ $\rho$  = tasa de evaporación]

→ Todos los caminos pierden feromona. Favorece "olvidar" rutas malas.

3. **DEPÓSITO:**  $\tau(i,j) \leftarrow \tau(i,j) + \Delta\tau$  [ $\Delta\tau = 1/\text{longitud\_ruta}$ ]

→ Hormigas con rutas CORTAS depositan MÁS feromona.

❏ aco\_tsp.py

# ACO simplificado para TSP

import random, math

```
def aco_tsp(distancias, n_hormigas=20, n_iter=100, alpha=1, beta=2, rho=0.1):
```

```
    """
```

```
    distancias: matriz nxn de distancias entre ciudades
```

```
    alpha: peso de la feromona (importancia del historial)
```

```
    beta: peso de la distancia (importancia de la cercanía)
```

```
    rho: tasa de evaporación (0 < rho < 1)
```

```
    """
```

```
    n = len(distancias)
```

```
    # Inicializar feromonas uniformemente en todos los arcos
```

```
    tau = [[1.0] * n for _ in range(n)]
```

```
    mejor_ruta, mejor_long = None, float('inf')
```

```
    for _ in range(n_iter):
```

```
        rutas = []
```

```
        for _ in range(n_hormigas):
```

```
            # Cada hormiga construye una ruta completa
```

```
            inicio = random.randint(0, n-1)
```

```
            ruta = [inicio]
```

```
            visitadas = {inicio}
```

```
            while len(ruta) < n:
```

```
                i = ruta[-1]
```

```
                # Calcular probabilidades para cada ciudad no visitada
```

```
                probs = []
```

```
                for j in range(n):
```

```
                    if j not in visitadas:
```

```
                        feromona = tau[i][j] ** alpha
```

```
                        visibilidad = (1.0 / distancias[i][j]) ** beta
```

```

        probs.append((j, feromona * visibilidad))

    # Selección probabilística (ruleta)
    total = sum(p for _, p in probs)
    r = random.uniform(0, total)
    acum = 0
    for j, p in probs:
        acum += p
        if acum >= r:
            ruta.append(j)
            visitadas.add(j)
            break

    rutas.append(ruta)

# EVAPORACIÓN: reducir feromonas en todos los arcos
for i in range(n):
    for j in range(n):
        tau[i][j] *= (1 - rho)

# DEPÓSITO: las hormigas depositan feromona en sus rutas
for ruta in rutas:
    longitud = sum(distancias[ruta[k]][ruta[k+1]] for k in range(n-1))
    longitud += distancias[ruta[-1]][ruta[0]] # vuelta al inicio
    delta_tau = 1.0 / longitud # rutas cortas → más feromona
    for k in range(n-1):
        tau[ruta[k]][ruta[k+1]] += delta_tau
        tau[ruta[k+1]][ruta[k]] += delta_tau # simétrico
# Actualizar mejor ruta global
if longitud < mejor_long:
    mejor_long, mejor_ruta = longitud, ruta

return mejor_ruta, mejor_long

```

## 7.4 Optimización por Enjambre de Partículas (PSO)

Inspirado en el comportamiento coordinado de bandadas de aves o cardúmenes. Cada **partícula** es una solución candidata que se mueve en el espacio continuo guiada por su mejor posición personal y la mejor posición global del enjambre.

### Memoria individual y colectiva:

$p_i$  (pbest) = mejor posición PERSONAL que ha visitado la partícula  $i$   
 $g$  (gbest) = mejor posición GLOBAL encontrada por CUALQUIER partícula

### Actualización de velocidad y posición en cada iteración:

$v \leftarrow w \cdot v + c_1 \cdot r_1 \cdot (pbest - x) + c_2 \cdot r_2 \cdot (gbest - x)$   
 $x \leftarrow x + v$   
 $w$  = inercia (qué tanto mantiene su dirección actual)  
 $c_1$  = componente cognitivo (atracción hacia su mejor personal)  
 $c_2$  = componente social (atracción hacia el mejor global)  
 $r_1, r_2$  = números aleatorios en  $[0,1]$  (stochastic)

```

❏ pso.py
# Particle Swarm Optimization (PSO) – espacio continuo
import random

def pso(funcion_objetivo, n_dims, n_particulas=30, n_iter=200,
        w=0.7, c1=1.5, c2=1.5, rango=(-10, 10)):
    """
    funcion_objetivo: f(x) a MAXIMIZAR (lista de n_dims floats → float)
    n_dims:           dimensiones del espacio de búsqueda
    w:                inercia    (momentum de la velocidad actual)
    c1:               cognitivo (atracción hacia pbest personal)
    c2:               social    (atracción hacia gbest global)
    rango:            (mín, máx) de cada dimensión
    """
    lo, hi = rango

    # Inicializar posiciones y velocidades aleatorias
    posiciones = [[random.uniform(lo, hi) for _ in range(n_dims)] for _ in range(n_particulas)]
    velocidades = [[random.uniform(-1, 1) for _ in range(n_dims)] for _ in range(n_particulas)]

    # pbest: mejor posición personal de cada partícula
    pbest = [p[:] for p in posiciones]
    pbest_val = [funcion_objetivo(p) for p in posiciones]

    # gbest: mejor posición global del enjambre
    gbest_idx = max(range(n_particulas), key=lambda i: pbest_val[i])
    gbest = pbest[gbest_idx][:]

    for _ in range(n_iter):
        for i in range(n_particulas):
            for d in range(n_dims):
                r1 = random.random() # componente aleatoria cognitiva
                r2 = random.random() # componente aleatoria social

                # Actualizar velocidad: inercia + cognitivo + social
                velocidades[i][d] = (w * velocidades[i][d]
                                     + c1 * r1 * (pbest[i][d] - posiciones[i][d])
                                     + c2 * r2 * (gbest[d] - posiciones[i][d]))

                # Actualizar posición y limitar al rango permitido
                posiciones[i][d] = max(lo, min(hi, posiciones[i][d] + velocidades[i][d]))

            # Evaluar nueva posición
            val = funcion_objetivo(posiciones[i])
            if val < pbest_val[i]:
                pbest[i] = posiciones[i][:] # Actualizar memoria personal
                pbest_val[i] = val
            if val < funcion_objetivo(gbest):
                gbest = posiciones[i][:] # Actualizar memoria global

    return gbest, funcion_objetivo(gbest)

```

## 7.5 Comparativa de Algoritmos Inspirados en la Naturaleza

| Algoritmo           | Inspiración                | Población     | Espacio             | Clave conceptual                                      |
|---------------------|----------------------------|---------------|---------------------|-------------------------------------------------------|
| Simulated Annealing | Metalurgia / recocido      | No (1 estado) | Discreto o continuo | Acepta movimientos malos con prob. $e^{(\Delta E/T)}$ |
| Algoritmo Genético  | Evolución darwiniana       | Sí            | Discreto (strings)  | Crossover + mutación → diversidad genética            |
| ACO                 | Comportamiento de hormigas | Sí            | Discreto (grafos)   | Feromona: refuerzo de buenas rutas + evaporación      |
| PSO                 | Bandadas / cardúmenes      | Sí            | Continuo            | pbest + gbest guían la velocidad de cada partícula    |